

Azure Service Bus – Creating a Namespace, Queue and Sending Messages

1. Creating Azure Service Bus Namespace

To work with Azure Service Bus, the first step is to create a **Service Bus Namespace**.

A Namespace acts as a container for Service Bus resources such as:

- Queues
- Topics
- Subscriptions

Basic Flow

```
Azure Portal
  |
  v
Service Bus
  |
  v
Create Namespace
  |
  v
Create Queue / Topic
```

2. Steps to Create a Service Bus Namespace

Step 1 – Open Azure Portal

Open the **Azure Portal** and search for:

Service Bus

Select **Service Bus** from the Azure services.

Step 2 – Create Namespace

Click:

Create → Namespace

A Namespace is required before creating Queues or Topics.

Step 3 – Select Resource Group

Select an existing **Resource Group** or create a new one.

Example:

Resource Group: MyResourceGroup

A Resource Group is a logical container used to organize Azure resources.

Step 4 – Enter Namespace Name

Give a unique name to the Service Bus Namespace.

Example:

first-service-bus-demo

The name must follow Azure's naming requirements and be unique as required by the service.

Step 5 – Select Region

Select the Azure region where the Service Bus Namespace should be created.

Example:

Central India

The region should generally be selected based on factors such as:

- Application location

- Latency
- Compliance requirements
- Availability requirements

3. Select Pricing Tier

Azure Service Bus has different pricing tiers.

Common tiers are:

- **Basic**
- **Standard**
- **Premium**

For this demonstration, we select:

Basic

General Understanding

Tier	Typical Usage
Basic	Basic messaging / learning / development scenarios
Standard	Development, QA, testing and general workloads
Premium	Production and advanced enterprise workloads

Important: Always verify the current Azure Service Bus feature availability and pricing before choosing a tier because Microsoft can change service capabilities and pricing.

4. Create the Namespace

After entering the required information:

Review + Create

Then click:

Create

Azure will deploy the Service Bus Namespace.

After deployment is completed:

[Go to Resource](#)

You can now open the newly created Service Bus Namespace.

5. Important Point – Namespace Alone Is Not Enough

After creating the Namespace, we have only created the **container**.

We have not yet created a Queue or Topic.

The structure is:

```
Service Bus Namespace
|
+----- Queue
|
+----- Topic
```

We need to create a messaging entity before we can send messages.

6. Creating a Queue

Inside the Service Bus Namespace:

```
Queue
↓
Create
```

Give the Queue a name.

Example:

```
our-first-queue
```

Then create the Queue.

7. Queue Configuration Options

When creating a Queue, Azure provides several configuration options.

Some important options include:

- Maximum delivery count
- Queue size
- Message Time To Live (TTL)
- Lock duration
- Enable dead-lettering on message expiration
- Enable partitioning

For the initial learning example, we can keep the default values.

We will study these settings separately because each one has an important purpose.

8. Maximum Delivery Count

Maximum Delivery Count specifies how many times a message can be delivered for processing before it is moved to the **Dead-Letter Queue**, depending on the queue's configuration.

Example:

```
Maximum Delivery Count = 5
```

If a message repeatedly fails processing and reaches the configured delivery limit, it can be moved to the Dead-Letter Queue.

We will study this in detail when learning about **Dead-Lettering**.

9. Queue Size

Queue size represents the amount of message storage available for the Queue.

Conceptually:

```
Queue
+-----+
| Message 1 |
| Message 2 |
| Message 3 |
| Message 4 |
+-----+
```

Messages are stored in the Queue until they are received or otherwise expire according to the configured behavior.

10. Message Time To Live (TTL)

Time To Live (TTL) defines how long a message can remain in the Queue before it expires.

Example:

```
Message
|
| TTL = configured duration
|
v
Message expires
```

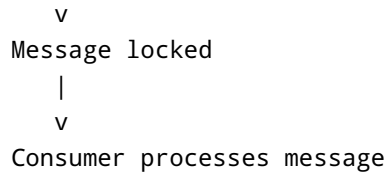
TTL is useful when a message is no longer useful after a certain amount of time.

11. Lock Duration

When a consumer receives a message using the appropriate receive mode, Azure Service Bus can place a lock on the message so another consumer does not process the same message simultaneously.

Conceptually:

```
Message
|
v
Consumer receives message
|
```



Lock duration determines how long that lock is held before it expires.

This becomes important when learning about:

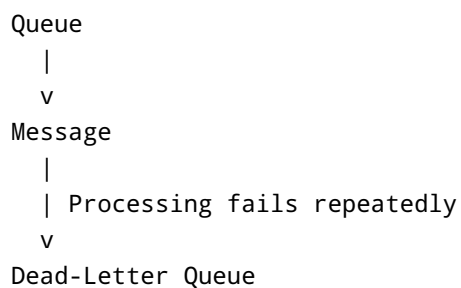
- Peek-Lock
- Message completion
- Abandoning messages
- Lock renewal

12. Dead-Lettering

Azure Service Bus supports a **Dead-Letter Queue (DLQ)**.

The Dead-Letter Queue is used to store messages that cannot be successfully processed or that meet certain dead-letter conditions.

Example:



This allows applications to investigate problematic messages separately.

13. Partitioning

Partitioning allows messages to be distributed across multiple message brokers for improved scalability and availability characteristics.

The exact behavior depends on the Service Bus configuration and entity type.

For basic learning, remember:

Partitioning can help Service Bus distribute messaging workload and improve scalability.

We can study partitioning separately.

14. Creating the Queue

For this example:

Queue Name :
our-first-queue

Keep the other settings as default.

Then click:

Create

The Queue is now created.

The structure becomes:

```
Service Bus Namespace
|
+---- our-first-queue
```

15. Opening the Queue

After the Queue is created, you can see it inside the Service Bus Namespace.

Click the Queue:

our-first-queue

This opens the Queue's configuration and management page.

16. Service Bus Explorer

Azure Portal provides **Service Bus Explorer** functionality that can be used to work with messages in a Queue or other supported messaging entities.

It can be used for tasks such as:

- Sending test messages
- Receiving/peeking messages
- Inspecting messages
- Testing Service Bus configuration

For learning and development, this is very useful because we can test Service Bus without writing code first.

17. Sending the First Message

Open the Queue and select:

Service Bus Explorer

Then select:

Send messages

Enter a message.

Example:

Hello Service Bus Queue

Then click:

Send

The message is now sent to the Queue.

18. Message Flow

The complete flow is:

```
Azure Portal
  |
  v
Service Bus Namespace
  |
  v
Queue
  |
  | Send Message
  v
"Hello Service Bus Queue"
```

The message is now stored in the Queue until it is received/processed or otherwise expires.

19. Reading the Message

To inspect the message, use the receive/peek functionality in **Service Bus Explorer**.

For example, you may select:

```
Peek from start
```

Then select the message.

You should be able to see:

```
Hello Service Bus Queue
```

This confirms that the message was successfully sent to the Queue.

20. Peek vs Receive – Basic Idea

This is an important concept that will be covered in more detail later.

Peek

Peek allows you to **look at a message without removing it from the Queue**.

```
Queue
|
| Peek
v
Read message
|
v
Message remains in Queue
```

Receive

Receiving a message means the consumer is obtaining the message for processing.

Depending on the receive mode, the message may be locked and later completed, abandoned, or otherwise handled.

```
Queue
|
| Receive
v
Consumer
|
v
Process message
```

21. Receive Modes

Azure Service Bus supports different message receiving behaviors.

The two important concepts to learn are:

Peek-Lock

The consumer receives the message under a lock.

The application can then:

Complete
Abandon
Defer
Dead-letter

depending on the processing outcome and API being used.

Receive and Delete

The message is removed from the Queue when it is received.

This means there is less opportunity to recover the message if processing fails after receipt.

Interview Tip

Peek-Lock is generally preferred when reliable message processing is important because the message is locked while being processed and can be completed only after successful processing.

22. Complete End-to-End Flow

The complete process from the example is:

```
Step 1
Azure Portal
  |
  v

Step 2
Create Resource Group
  |
  v

Step 3
Create Service Bus Namespace
  |
  v

Step 4
Select Pricing Tier
  |
  v
```

Step 5
Create Queue
|
v

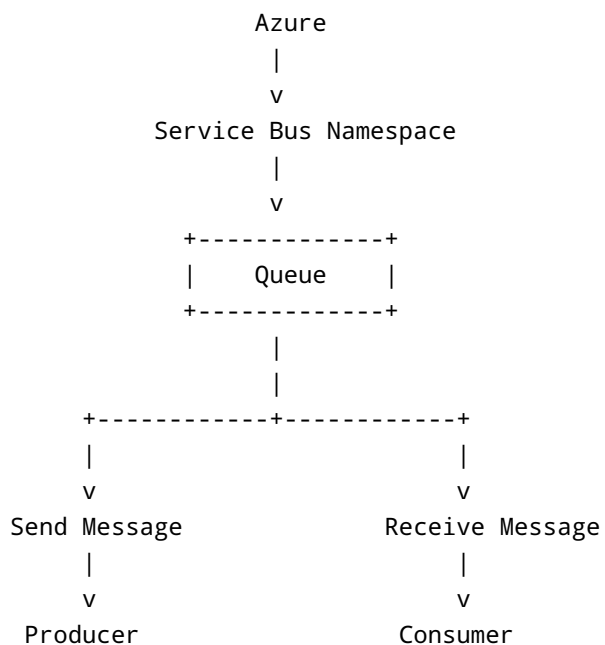
Step 6
Open Service Bus Explorer
|
v

Step 7
Send Message
|
v

Step 8
Message Stored in Queue
|
v

Step 9
Peek / Receive Message

23. Complete Architecture



Using the Azure Portal:

```
Azure Portal
  |
  v
Service Bus Namespace
  |
  v
our-first-queue
  |
+---- Send Message
  |
+---- Peek / Receive Message
```

24. Important Interview Questions

Q1. What is the first resource we create in Azure Service Bus?

Answer:

We typically create a **Service Bus Namespace** first. The Namespace acts as a container for messaging entities such as Queues and Topics.

Q2. Can we create a Queue without a Namespace?

Answer:

No. A Queue belongs to a Service Bus Namespace.

Q3. What is Service Bus Explorer?

Answer:

Service Bus Explorer is a tool available through Azure tooling that allows us to interact with Service Bus messaging entities, such as sending, peeking, and receiving messages.

Q4. How do you send a test message to a Queue?

Answer:

Open the Queue in Azure Portal, open Service Bus Explorer, select the send-message option, enter the message body, and send it.

Q5. How can you inspect a message without removing it?

Answer:

Use **Peek**. Peek allows us to inspect the message without consuming it.

Q6. What is Maximum Delivery Count?

Answer:

Maximum Delivery Count specifies the maximum number of delivery attempts for a message before it can be moved to the Dead-Letter Queue.

Q7. What is Message TTL?

Answer:

Message Time To Live specifies how long a message can remain available before it expires.

Q8. What is Dead-Letter Queue?

Answer:

A Dead-Letter Queue stores messages that cannot be successfully processed or meet conditions that cause them to be dead-lettered.

25. Things to Remember

Namespace

Container for Service Bus resources

Queue

Used for messaging

Service Bus Explorer

Used to test and interact with messages

Send

Producer → Queue

Peek

Look at message without consuming it

Receive

Get message for processing

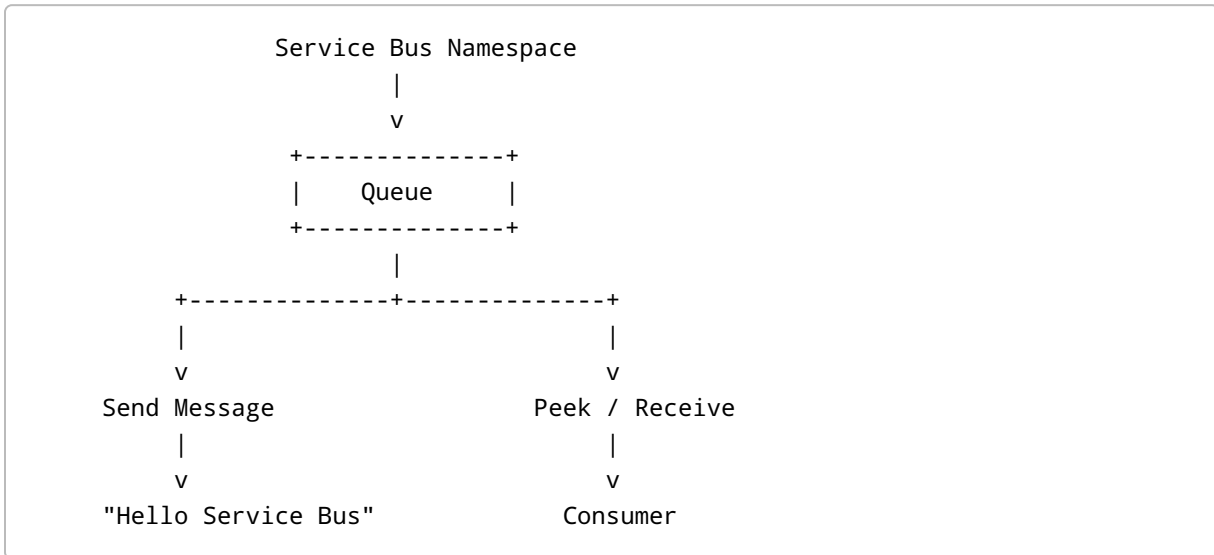
Dead-Letter Queue

Problematic / dead-lettered messages

TTL

How long a message remains available

26. Quick Revision Diagram



Remember the Flow:

Namespace → Queue → Send → Store → Peek/Receive → Process

27. Important Note About Pricing Tiers

For interview preparation, remember the basic idea:

```
Basic
  ↓
Standard
  ↓
Premium
```

However, don't memorize statements such as **"Basic is always for local, Standard is always for QA, Premium is always for production."**

Pricing tiers are selected based on **required features, performance, scale, isolation, and workload requirements**.

For example:

- Development may use Basic or Standard depending on requirements.
- QA may use Standard or another appropriate tier.
- Production may require Standard or Premium depending on the workload and required capabilities.

Always check the current Azure documentation for the latest feature availability and pricing before making a real deployment decision.

Final Interview Summary

To use Azure Service Bus, we first create a Service Bus Namespace. Inside the Namespace, we can create messaging entities such as Queues and Topics. A Queue is commonly used for one-to-one messaging. We can use Service Bus Explorer to send and inspect test messages. A message can be peeked without consuming it, while receiving a message is part of the message-processing flow. Queue settings such as Maximum Delivery Count, TTL, Lock Duration, Dead-Lettering, and Partitioning control different aspects of message processing and reliability.

Most Important Flow

```
Resource Group
  ↓
Service Bus Namespace
  ↓
Queue
  ↓
Service Bus Explorer
  ↓
Send Message
  ↓
Message Stored
  ↓
Peek / Receive
  ↓
Process Message
```